Assignment
Secure AI Systems – Red and Blue Teaming an MNIST Classifier

Secure Software Engineering

Shradha Sanjaykant Suryawanshi CS25MTECH12018
Suresh Kumar Baddipudi CS25M26P140002

Repository: github.com/Shradha-iith-12018/Assignment_Secure_AI_Systems

Primary reproducible result tag: `cpu-canonical-v1`
Independent reproduction tag: `cpu-shradha-v1`

October 23, 2026

Contents

1 Introduction **2**

2 System and Experimental Setup **2**
2.1 Classifier . . . 2
2.2 Primary software environment . . . 3
2.3 Data flow and trust boundaries . . . 3

3 Baseline Performance **3**

4 Threat Modeling **4**
4.1 Assets . . . 4
4.2 Attacker model . . . 5
4.3 STRIDE mapped to OWASP Machine Learning Top 10 . . . 5
4.4 LINDDUN privacy discussion . . . 6

5 Secure Software Engineering and SAST **7**
5.1 Tools and scope . . . 7
5.2 Application hardening represented in the final code . . . 8

6 Red Teaming **9**
6.1 Attack 1: FGSM and PGD evasion (OWASP ML01) . . . 9
6.2 Attack 2: backdoor data poisoning (OWASP ML02) . . . 9

7 Blue Teaming **10**
7.1 Adversarial training . . . 10
7.2 Feature squeezing . . . 10
7.3 Activation clustering . . . 11
7.4 Secure model artifact handling . . . 11

8 Comparative Evaluation **12**

9 Independent Reproducibility Check **12**

10 Limitations and Threats to Validity **13**

11 Contribution Statement **13**

12 Conclusion and Observations **13**

13 Reproducing the Results **14**

1. Introduction

This project applies secure software engineering and machine-learning security concepts to an MNIST handwritten-digit classifier. The assignment requires a convolutional neural network (CNN), clean performance evaluation, STRIDE threat modeling with OWASP Machine Learning Top 10 mappings, LINDDUN privacy analysis, static application security testing (SAST), red-team attacks, blue-team defenses, and comparative evaluation.

The implementation is maintained in the public GitHub repository github.com/Shradha-iith-12018/Assignment_Secure_AI_Systems. The final repository contains the CNN implementation, attack and defense code, generated adversarial and poisoned datasets, trained model artifacts, SAST evidence, figures, and machine-readable result files. Two complete CPU executions were preserved with Git tags. The primary result set used throughout this report is `cpu-canonical-v1`; `cpu-shradha-v1`is used as an independent reproducibility check.

Table 1: Repository structure.

Path	Contents
`src/`	Shared utilities, CNN definition, validation, training, evaluation metrics, and result summary generation.
`attacks/`	FGSM/PGD evasion and backdoor data poisoning.
`defenses/`	Adversarial training, feature squeezing, and activation clustering.
`sast/`	SAST runner and a deliberately insecure control sample.
`scripts/`	Environment capture and supporting scripts.
`results/`	JSON metrics, summary, figures, environment metadata, and SAST reports.
`models/`	`.safetensors` weights and SHA-256 manifest.
`data/generated/`	Saved adversarial, poisoned, and triggered datasets.

2. System and Experimental Setup

2.1 Classifier

The application classifies 28×28 grayscale MNIST digits (60,000 training images and 10,000 test images). Pixels are represented as floating-point values in $[0, 1]$; normalization using mean 0.1307 and standard deviation 0.3081 occurs inside the network so that attacks and defenses operate in ordinary pixel space.

Table 2: Model and training configuration.

Item	Configuration
Architecture	$\text{Conv}(1 \to 32, 3 \times 3) \to \text{ReLU} \to \text{Conv}(32 \to 64, 3 \times 3) \to \text{ReLU} \to \text{MaxPool}$ $2 \times 2 \to \text{Dropout } 0.25 \to \text{FC}(9216 \to 128) \to \text{ReLU} \to \text{Dropout } 0.5 \to$ $\text{FC}(128 \to 10)$.
Optimizer	Adam, learning rate 10^{-3}.
Loss	Cross-entropy.
Batch size	128.
Epochs	5.
Seed	42 for Python, NumPy, and PyTorch.
Execution	CPU for both preserved runs; device selection is controlled through `SSE_DEVICE=cpu|cuda|auto`.

2.2 Primary software environment

The canonical run used Python 3.10.20, PyTorch 2.11.0, torchvision 0.26.0, NumPy 2.2.6, scikit-learn 1.7.2, matplotlib 3.10.8, IBM Adversarial Robustness Toolbox (ART) 1.20.1, safetensors 0.8.0, Bandit 1.9.4, Semgrep 1.181.0, and pip-audit 2.10.1. The execution device was explicitly CPU. The environment is recorded in `results/environment.json`.

2.3 Data flow and trust boundaries

Figure 1 shows the system data flow and the three trust boundaries at which external data, stored model artifacts, and inference requests cross into trusted components.

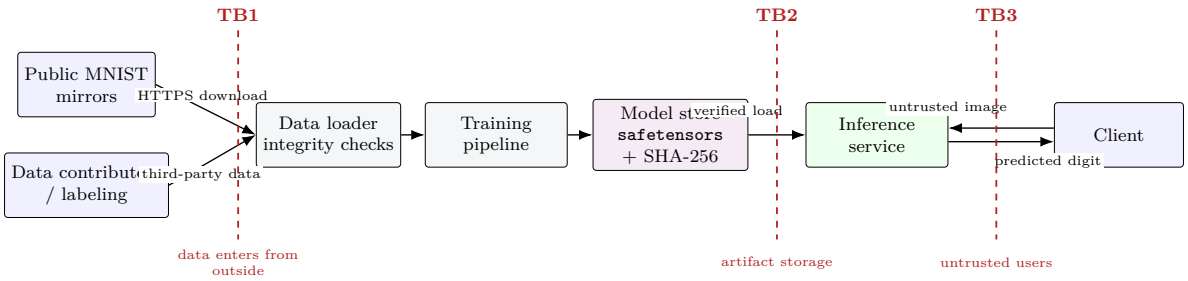


Figure 1: Data flow and trust boundaries in the MNIST classification system.

- **TB1:** External data enters through dataset downloads or third-party contributions.
- **TB2:** Trained model artifacts cross from storage into the inference process and must be integrity-checked before loading.
- **TB3:** Untrusted users submit inference inputs and receive predictions.

3. Baseline Performance

The primary baseline model was trained for five epochs and evaluated on the complete 10,000-image MNIST test set. Table 3 reports the canonical metrics.

Table 3: Canonical baseline performance (`cpu-canonical-v1`).

Metric	Value
Accuracy	99.10%
Cross-entropy loss	0.0304
Macro F1	0.9909
Single-image median latency	0.970 ms/image
Batched inference	0.6153 ms/image
Training time	348 s

The training losses by epoch were 0.2378, 0.0846, 0.0651, 0.0538, and 0.0433. The high clean accuracy establishes a strong functional baseline but does not demonstrate robustness to adversarial or poisoned inputs.

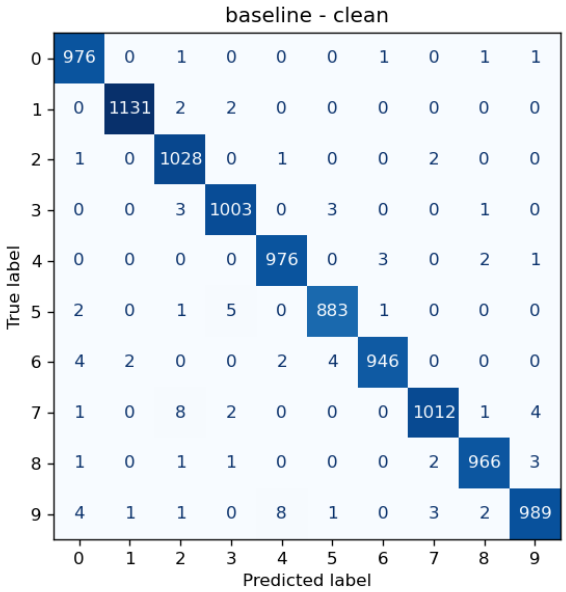


Figure 2: Baseline confusion matrix for the canonical CPU run.

4. Threat Modeling

4.1 Assets

Table 4: Security-relevant assets.

Asset	**Why it matters**
Training data	Determines learned behavior; poisoning can persist into every model trained from the corrupted data.
Model weights	Intellectual property and executable decision logic; unauthorized modification changes predictions.
Inference service	Must preserve prediction correctness and availability.
Predictions/confidence scores	Can leak information about the model or training data.
Training/inference code and dependencies	Compromise can provide control over data, artifacts, or execution.
Generated adversarial/poisoned datasets	Security-test evidence that must remain aligned with the evaluated model and experiment configuration.

4.2 Attacker model

Table 5: Attacker model.

Attacker	Goal	Knowledge / capability	Access point	Status
A1 Evasion attacker	Cause misclassification of selected inputs.	White-box access to architecture/weights; can perturb each pixel within $\epsilon = 0.2$ in L_∞.	TB3	Tested
A2 Data poisoner	Plant trigger → target label 0 while preserving clean accuracy.	Can insert/modify up to 3% of training samples and labels.	TB1	Tested
A3 Artifact tamperer	Replace model artifacts to alter behavior or abuse unsafe loading.	Write access to the model store or download location.	TB2	Mitigated by design
A4 Black-box API user	Extract the model or infer training membership.	Query-only access to model outputs.	TB3	Discussed, not tested

Out of scope are an insider with unrestricted access to the training host/source code and physical attacks on hardware.

4.3 STRIDE mapped to OWASP Machine Learning Top 10

The model follows the ML-specific interpretation of STRIDE and maps the relevant threats to the OWASP Machine Learning Security Top 10 [1].

Table 6: STRIDE and OWASP ML mapping.

STRIDE	ML-specific threat	OWASP ML risk	Project treatment
Spoofing	Adversarial input appears valid to a human while changing the model's perception.	ML01 Input Manipulation	FGSM/PGD attacked; adversarial training and squeezing defended.
	Fake dataset/model source presented as genuine.	ML06 AI Supply Chain	Dataset integrity checks; SHA-256 model manifest.
Tampering	Relabeled or trigger-stamped training samples.	ML02 Data Poisoning	Backdoor poisoning attacked; activation clustering defended.
	Modified model weights.	ML10 Model Poisoning	SHA-256 verification and non-pickle safetensors.

STRIDE	ML-specific threat	OWASP ML risk	Project treatment
	Prediction altered between model and consumer.	ML09 Output Integrity	Out of scope; production system should authenticate responses.
Repudiation	Weak provenance for data/model versions can make incidents difficult to trace.	ML02, ML10	Seeded runs, hashes, Git tags, and retained artifacts provide partial provenance.
Information disclosure	Membership inference or model inversion from confidence outputs.	ML04 Membership Inference; ML03 Model Inversion	Discussed; mitigations include label-only output and privacy-preserving training.
	Model extraction from repeated queries.	ML05 Model Theft	Discussed; rate limiting and monitoring recommended.
Denial of service	Broad poisoning/model skewing or query flooding.	ML02, ML08 Model Skewing	Not directly tested; validation and service-layer rate limiting recommended.
Elevation of privilege	Unsafe deserialization or malicious model artifact triggers code execution.	ML06, ML10	Earlier `torch.load` path removed; current implementation uses safetensors plus SHA-256 verification.
	Backdoored transferred model.	ML07 Transfer Learning Attack	Not applicable because the model is trained from scratch.

4.4 LINDDUN privacy discussion

MNIST itself is a public benchmark and this project does not treat it as personal data. A realistic deployment, however, could classify digits cropped from bank cheques, postal forms, tax forms, or account documents. In that setting, LINDDUN identifies meaningful privacy risks.

Table 7: LINDDUN privacy considerations for a real-world deployment.

Category	Threat	Representative mitigation
Linking	Handwriting style or retained embeddings can link records from the same writer.	Avoid retaining raw inputs/embeddings; aggregate logs.
Identifying	Digit crops may reveal account numbers, addresses, amounts, or training membership.	Data minimization; label-only outputs; differential privacy where appropriate.
Non-repudiation	Detailed logs can bind a person to a transaction.	Pseudonymization and retention limits.
Detecting	Query metadata can reveal that a person submitted a document.	Access control and minimal per-user telemetry.
Data disclosure	Training images/logs can be exposed or over-retained.	Encryption at rest, strict retention, and supplier controls.
Unawareness	Writers may not know handwriting is reused for training.	Notice, consent/lawful basis, and opt-out where appropriate.
Non-compliance	Personal-document processing can violate purpose or storage limitations.	DPIA, documented lawful basis, deletion schedule, and governance.

5. Secure Software Engineering and SAST

5.1 Tools and scope

The application was scanned using Bandit, Semgrep, and pip-audit. Bandit is an OWASP-listed Python SAST tool; Semgrep adds rulesets including security-audit and Trail of Bits ML/security checks; pip-audit provides dependency vulnerability analysis. The final application scope includes `src/`, `attacks/`, `defenses/`, and `scripts/`.

Table 8: Final SAST/dependency-audit result.

Tool	Application result	Control sample
Bandit 1.9.4	0 findings	Findings detected (expected)
Semgrep 1.181.0	0 blocking findings; 277 rules / 25 targets	3 blocking findings (expected)
pip-audit 2.10.1	0 known vulnerabilities	N/A

The deliberately insecure control sample under `sast/control/` is never imported or executed. Its purpose is to prove that the configured scanners can detect concrete unsafe patterns instead of treating a clean application scan as sufficient evidence by itself. Bandit reports nine findings on this control file (1 high, 4 medium, 4 low), while Semgrep reports three blocking findings.

Table 9: Bandit findings on the deliberately insecure control sample and corresponding safe patterns in the application.

Rule	Severity	Control pattern	How the application avoids the pattern
B614	Medium	Unsafe `torch.load`	Current model artifacts use non-pickle `safetensors`; SHA-256 is verified before loading.
B301	Medium	`pickle.load`	The application does not deserialize model/data artifacts with `pickle`; generated NPZ files are loaded with `allow_pickle=False`.
B403	Low	Importing `pickle`	Application code does not import `pickle` for artifact loading.
B602	High	Shell command with `shell=True`	Application code does not invoke shell commands for dataset extraction or experiment execution.
B404	Low	Importing `subprocess`	The application does not require `subprocess`; environment versions are read through `importlib.metadata`.
B310	Medium	Dynamic URL retrieval	MNIST is obtained through `torchvision.datasets.MNIST`, which uses known dataset mirrors and integrity checks.
B108	Medium	Hard-coded temporary path	Project artifacts are written under controlled project directories such as `data/`, `models/`, and `results/`, not a fixed `/tmp` path.
B311	Low	Non-cryptographic randomness	The control sample uses `random.sample`; experiment sampling in the application is reproducibility-oriented and uses seeded RNGs rather than security tokens.
B101	Low	`assert` for validation	Trust-boundary validation uses explicit exceptions such as `ValueError` and `TypeError`, which are not removed by optimized Python execution.

Table 10: Semgrep findings on the deliberately insecure control sample and corresponding safe patterns in the application.

Rule	Severity	Control pattern	How the application avoids the pattern
`pickles-in-pytorch`	Error	Unsafe PyTorch deserialization	Model persistence uses `safetensors` plus SHA-256 integrity verification rather than pickle-backed loading.
`avoid-pickle`	Warning	`pickle.load`	No pickle deserialization is used for application artifacts; NPZ loading explicitly disables pickle.
`dynamic-urllib-use-detected`	Warning	User-controlled `urllib` URL	Dataset download is delegated to `torchvision.datasets.MNIST` rather than accepting an arbitrary runtime URL.

5.2 Application hardening represented in the final code

The final repository is clean under the configured Bandit and Semgrep application scans, but the Git history does not support claiming that every control-sample finding was first discovered by SAST and then remediated. In particular, the initial committed model loader already used `torch.load(..., weights_only=True)` with SHA-256 verification. The later migration to `safetensors` is therefore presented as an additional hardening step that removes pickle-backed model serialization entirely, not as a documented response to a committed Semgrep finding. Likewise, the committed environment-recording script uses `importlib.metadata` and contains no `subprocess` call; without separate scan evidence, no historical Bandit remediation claim is made for that helper.

The final implementation nevertheless applies several concrete secure-engineering controls: non-pickle `safetensors` model artifacts; SHA-256 integrity verification; safe artifact-name validation; explicit validation of MNIST shapes, pixel ranges, labels, attack parameters, and generated datasets; NPZ loading with `allow_pickle=False`; reproducible seeds; and robust CPU/GPU selection with an actual CUDA execution probe.

SAST remains intentionally narrow: it can identify code-level weaknesses but cannot discover adversarial examples, training-data poisoning, backdoor behavior, or privacy leakage from model outputs. Those risks require ML-specific testing.

6. Red Teaming

6.1 Attack 1: FGSM and PGD evasion (OWASP ML01)

The first attack targets inference-time integrity at TB3. White-box, untargeted adversarial examples are generated with ART using an L_∞ budget of $\epsilon = 0.2$. FGSM [2] uses one gradient-sign step. PGD [3] uses 40 projected iterations with step size 0.02, making it a substantially stronger iterative attacker under the same perturbation budget.

Table 11: Canonical evasion results against the baseline model.

Test set	Accuracy	Loss	Macro F1
Clean	99.10%	0.0304	0.9909
FGSM, $\epsilon = 0.2$	49.09%	1.7922	0.4877
PGD, $\epsilon = 0.2$	0.89%	12.7718	0.0090

FGSM cuts accuracy roughly in half, while PGD reduces accuracy to below 1%. The result demonstrates that excellent clean performance does not imply adversarial robustness.

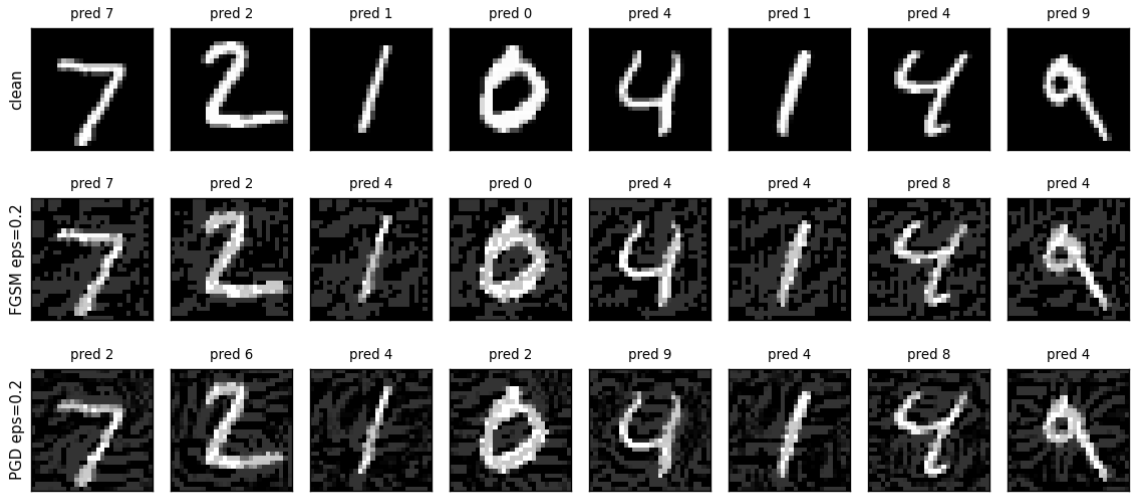


Figure 3: Representative clean, FGSM, and PGD examples generated by the repository pipeline. The perturbations preserve human readability while changing model predictions.

The adversarial datasets are stored as `data/generated/adv_fgsm_test_baseline.npz` and `data/generated/adv_pgd_test_baseline.npz`.

6.2 Attack 2: backdoor data poisoning (OWASP ML02)

The second attack models a compromised contributor or data source at TB1. A 4×4 white trigger is stamped near the bottom-right corner of 1,800 training images (3% of 60,000), excluding samples already in the target class, and the poisoned samples are relabeled to target digit 0. The attack follows the backdoor threat demonstrated by BadNets-style poisoning [4].

Table 12: Canonical backdoor poisoning results.

Model	Clean accuracy	Triggered accuracy	Attack success rate
Baseline	99.10%	98.97%	0.11%
Poisoned	98.94%	0.01%	99.99%

The poisoned model retains near-baseline clean accuracy while mapping almost every triggered non-zero digit to class 0. This is dangerous precisely because normal validation data does not reveal the implanted behavior.

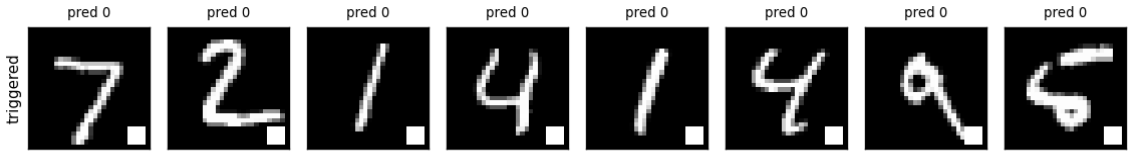


Figure 4: Representative triggered inputs from the poisoning experiment.

The generated poisoning artifacts are saved under `data/generated/` and the poisoned model is stored as `models/poisoned.safetensors`.

7. Blue Teaming

7.1 Adversarial training

The principal evasion defense retrains a fresh model for five epochs. In each minibatch, approximately half the samples are replaced with PGD adversarial examples generated against the model in its current state. The training attack uses $\epsilon = 0.2$, 10 iterations, step size 0.05, and random initialization. Evaluation includes both transfer attacks (saved adversarial examples from the baseline) and new white-box attacks against the defended model itself. The latter is the fairer robustness test because it models an attacker targeting the deployed defense.

Table 13: Canonical adversarial-training results.

Evaluation	Baseline	Adversarially trained
Clean accuracy	99.10%	98.78%
FGSM white-box	49.09%	96.05%
PGD white-box	0.89%	93.29%
FGSM transfer	–	97.14%
PGD transfer	–	97.09%
Training time	348 s	1478 s

The defense sacrifices only 0.32 percentage points of clean accuracy while recovering more than 92 percentage points under white-box PGD. The main cost is training time (approximately 4.24× the baseline on the canonical CPU system). Robustness is demonstrated only around the tested $\epsilon = 0.2$ threat model; it should not be generalized to arbitrary budgets or attacks.

7.2 Feature squeezing

Feature squeezing [6] reduces the degrees of freedom available to the adversary. The implementation evaluates 1-bit quantization and a 2×2 median filter. It also acts as a detector by

comparing softmax outputs before and after squeezing; the threshold is calibrated on 5,000 clean training images for a target 5% false-positive rate.

Table 14: Canonical feature-squeezing results on the baseline model.

Test set	**Raw**	**1-bit**	**Median 2x2**	**Flagged**
Clean	99.10%	98.91%	98.09%	6.78%
FGSM	49.09%	93.77%	61.36%	76.52%
PGD	0.89%	93.12%	15.65%	97.69%

One-bit squeezing is particularly effective on MNIST because pixel values are close to binary and the $\epsilon = 0.2$ perturbation often disappears during rounding. This result is not directly comparable to adversarial training because the saved adversarial examples were crafted without adapting to the squeezer. An adaptive attacker can explicitly optimize through or around preprocessing.

7.3 Activation clustering

Activation clustering [5] defends the training-data path. The poisoned model's 128-dimensional penultimate activations are collected, reduced with PCA, and clustered per label using k-means ($k = 2$). A small, well-separated cluster is considered suspicious. The implementation flags a class cluster when its fraction is below 0.35 and its silhouette score exceeds 0.35, removes suspicious samples, and retrains the model.

Table 15: Canonical activation-clustering defense.

Metric	**Value**
Flagged training samples	1,795
Detection precision	1.000
Detection recall	0.997
Sanitized clean accuracy	99.08%
Sanitized triggered accuracy	98.78%
Sanitized backdoor ASR	0.29%

The defense removes nearly all poisoned training samples without sacrificing clean accuracy and reduces the backdoor attack success rate from 99.99% to 0.29%. A limitation is that the clustering threshold was tuned with knowledge of the experimental poisoning pattern; a production system should validate thresholds using known-clean data and consider more diverse or distributed triggers.

7.4 Secure model artifact handling

The final artifact-loading path addresses A3 and ML supply-chain/model-poisoning concerns. Models are serialized using `safetensors`, which stores tensors without Python pickle execution. The implementation validates a restricted artifact name, computes SHA-256 over the model file, compares it with `models/manifest.json`, loads tensors on CPU, validates the state dictionary against the expected model architecture, and only then moves the model to the selected device.

The manifest is an integrity reference, not a digital signature. An attacker with write access to both the model file and manifest could alter both; a production deployment should keep the trust root separately protected or use signed artifacts/model-registry attestations.

8. Comparative Evaluation

Table 16: Security-performance comparison using the canonical run.

Configuration	Clean	FGSM	PGD	Backdoor ASR / note
Baseline	99.10%	49.09%	0.89%	0.11% on naturally triggered baseline
Baseline + 1-bit squeeze	98.91%	93.77%	93.12%	Non-adaptive evasion preprocessing
Adversarially trained	98.78%	96.05%	93.29%	Fresh white-box attacks
Poisoned	98.94%	–	–	99.99%
Sanitized after clustering	99.08%	–	–	0.29%

No single defense covers the entire threat model. Adversarial training and squeezing address A1; activation clustering addresses A2; safetensors plus SHA-256 verification addresses part of A3; A4 privacy/extraction threats remain open and require service- and privacy-layer controls.

9. Independent Reproducibility Check

A second team member independently executed the same source code on another CPU system and preserved the results under `cpu-shradha-v1`. A direct Git diff between the two tags for `src/`, `attacks/`, `defenses/`, `sast/`, `scripts/`, the requirements files, and `run_all.sh` produced no differences, establishing that the two result sets were generated by the same implementation.

Table 17: Independent reproducibility comparison.

Metric	`cpu-canonical-v1`	`cpu-shradha-v1`
Baseline clean accuracy	99.10%	98.98%
FGSM baseline accuracy	49.09%	54.26%
PGD baseline accuracy	0.89%	0.95%
Adversarially trained clean	98.78%	98.77%
Adversarially trained FGSM (white-box)	96.05%	96.28%
Adversarially trained PGD (white-box)	93.29%	93.32%
Poisoned clean accuracy	98.94%	99.08%
Backdoor attack success rate	99.99%	99.98%
Sanitized clean accuracy	99.08%	99.10%
Sanitized backdoor ASR	0.29%	0.91%
Poison detection precision	1.000	1.000
Poison detection recall	0.997	0.994

The environments differed substantially: the canonical run used Python 3.10.20 / PyTorch 2.11.0, while the independent run used Python 3.14.7 / PyTorch 2.14.1+cpu. The security conclusions remained the same despite these differences. Timing values differed more strongly

because they are hardware/runtime dependent; therefore timings were not averaged across systems.

10. Limitations and Threats to Validity

- The evaluation uses MNIST, whose low dimensionality and near-binary pixel distribution make some preprocessing defenses stronger than they would be on natural images.
- Evasion robustness is demonstrated for a specific L_∞ budget ($\epsilon = 0.2$) and selected FGSM and PGD settings, not for every adversarial attack.
- Feature squeezing was evaluated against non-adaptive saved adversarial samples; an attacker aware of the defense can adapt.
- The poisoning experiment uses one consistent visible 4×4 trigger and a single target class. Distributed, semantic, or clean-label poisoning may be harder to detect.
- Activation-clustering thresholds were informed by the experimental setting; deployment requires separate calibration.
- SHA-256 manifests provide integrity only when the manifest itself is trusted. Production systems should use stronger provenance and signing.
- Membership inference, model inversion, model theft, output-integrity attacks, service-level denial of service, and differential-privacy defenses were not experimentally evaluated.
- CPU timing is machine dependent. Accuracy/robustness results were independently reproduced, while timing was treated as environment-specific.

11. Contribution Statement

Shradha Suryawanshi developed and committed the initial end-to-end implementation, including the baseline CNN, red-team attacks, blue-team defenses, initial SAST pipeline, experiments, and first report. Suresh Kumar Baddipudi subsequently performed substantial security hardening and refactoring, improved reproducibility and device portability, strengthened secure-engineering controls and SAST evidence, migrated model serialization to `safetensors`, reran and validated the complete pipeline, generated the canonical result set and artifacts, and finalized repository documentation. Shradha then independently reproduced the complete pipeline on a second CPU system using the same source code and preserved a separately tagged result set. Both contributors participated in validation, interpretation of results, and preparation of the final report.

12. Conclusion and Observations

The experiment demonstrates that functional correctness and ML security are separate properties. The canonical baseline reached 99.10% clean accuracy yet fell to 49.09% under FGSM and 0.89% under PGD. Likewise, poisoning only 3% of training samples produced a model with 98.94% clean accuracy but a 99.99% backdoor attack success rate. A normal clean test set alone would therefore miss both major weaknesses.

Adversarial training was the strongest evaluated evasion defense, restoring white-box PGD accuracy to 93.29% while maintaining 98.78% clean accuracy. Feature squeezing offered a cheap secondary control and detector, but its strong MNIST performance depends on input characteristics and non-adaptive attacks. Activation clustering was highly effective against the

tested backdoor, detecting poisoned samples with precision 1.000 and recall 0.997 and reducing ASR to 0.29% after sanitized retraining.

The software-security layer is also important. The SAST control sample demonstrates that the configured scanners detect unsafe deserialization, shell execution, dynamic URL handling, temporary-file, weak-randomness, and validation patterns, while the final application scan is clean. The application also uses non-pickle safetensors artifacts, retains SHA-256 integrity verification, and validates data at trust boundaries. However, SAST cannot replace ML-specific red teaming: adversarial examples and poisoned data are semantically malicious while still looking like valid program inputs.

Finally, the second independent CPU execution reproduced the same qualitative conclusions using identical code but a different software environment. This strengthens confidence that the observed attack/defense behavior is not an artifact of a single machine. The overall lesson is that secure AI requires layered controls spanning data provenance, artifact integrity, adversarial robustness, monitoring, privacy, and conventional software security.

13. Reproducing the Results

From the repository root, create/activate a Python environment, install the runtime and development requirements, select the execution device, and run the pipeline:

```
pip install -r requirements.txt -r requirements-dev.txt
SSE_DEVICE=cpu ./run_all.sh
./sast/run_sast.sh
```

For a compatible CUDA system, `SSE_DEVICE=cuda` may be used. The preserved Git tags provide stable experiment snapshots:

```
git fetch --tags
git checkout cpu-canonical-v1
# or
git checkout cpu-shradha-v1
```

References

[1] OWASP Foundation, *Machine Learning Security Top 10*, 2023.

[2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” ICLR, 2015.

[3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” ICLR, 2018.

[4] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain,” 2017.

[5] B. Chen et al., “Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering,” 2018.

[6] W. Xu, D. Evans, and Y. Qi, “Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks,” NDSS, 2018.

[7] M.-I. Nicolae et al., “Adversarial Robustness Toolbox v1.0.0,” 2018.

[8] M. Deng et al., “A Privacy Threat Analysis Framework: Supporting the Elicitation and Fulfillment of Privacy Requirements,” Requirements Engineering, 2011.

[9] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.